

Nexus Independent Audit

Architecture, DDD, API design, runtime correctness, scaling, persistence, security, operations, packaging, documentation, examples, and product feasibility.

Verdict: a substantial actor-system foundation that should remain experimental until its durable, lifecycle, delivery, and security release gates are closed.

68

TOTAL FINDINGS

Audit date: 16 July 2026

Semantic baseline: 8970faa2

Prepared for technical and executive review

37

HIGH SEVERITY

31

MEDIUM SEVERITY

Nexus Independent Product and Engineering Audit

Audit date: 2026-07-16

Repository state: commit `8970faa2`, branch `chore/psalm-zero-suppressions`, identical to `origin/main` at audit start

Scope: architecture, DDD, public API and DSL, runtime correctness, scaling, persistence, security, operations, packaging, tests, documentation, examples, and product feasibility

Method: four independent review tracks, two adversarial peer-review passes, CodeGraph-assisted call-path analysis, direct source and documentation review, and executable verification in the repository's Docker environment

Concurrent-worktree note: the audit started with only `.codegraph/` and `.whet/` untracked. During the review, unrelated concurrent work appeared across at least 95 tracked files plus new tooling files. The auditors did not make those production changes and did not revert them. Targeted synthesis-time inspection found mostly Psalm-suppression removal, type annotations, dependency metadata, and tooling work, but the tree continued changing. The audit therefore separates immutable source evidence from dirty-worktree verification. Re-run every release gate on a committed candidate.

Contents

- Executive Verdict
- Independent Review Panels
- How Nexus Would Really Be Used
- Constrained Adopter Journey
- DDD Review
- DSL and API Review
- DSL Surface Scorecard
- Consolidated Findings
- Verification Record
- Prioritized Remediation Roadmap
- Release Gates
- Final Feasibility Assessment
- Evidence Appendix

Evidence Provenance

- **BASE:** commit `8970faa2` is the immutable semantic review baseline. Source findings are anchored to repository-relative paths and symbols at BASE unless explicitly marked DIRTY.
- **DIRTY:** test, static-analysis, dependency, and documentation commands ran against the concurrently changing working tree. Those results describe only that timestamped tree and must not be attributed to BASE or `origin/main`.
- **Reproduction:** inspect cited BASE source with `git show 8970faa2:<path> | nl -ba`. Line ranges are navigation aids; the cited symbol and behavior are authoritative when later edits shift lines.
- **Confidence:** principal High findings were independently challenged by another reviewer. Conditional deployment findings state their preconditions in the finding text rather than changing severity labels.

Independent Review Panels

Panel	Independent mandate	Primary review lens
Architecture, DDD, and DSL	Trace advertised abstractions through interpreters and examples; challenge domain boundaries and API truthfulness	Package boundaries, actor-per-aggregate, persistence effects, HTTP/app DSL, packaging.
Runtime, Scaling, and Operations	Assume saturation, partial failure, restart, and distribution; verify cross-runtime contracts	Mailboxes, lifecycle, supervision, timers, Messenger, worker threads, TCP cluster, recovery, operations.
Security	Build a trust-boundary and attack-surface model; seek fail-open defaults and unsafe data crossings	HTTP/WebSocket auth, JWT, cluster trust, serialization, DB pools, broker routing, supply chain.
Adversarial synthesis	Recheck evidence, severity, deployment preconditions, contradictions, and release decisions	Cross-panel deduplication, severity normalization, verification provenance, feasibility, and final release gates.

Panels reviewed independently before synthesis. A finding was retained only when it had a reproducible source path, executable observation, documentation contradiction, or explicit assurance gap. Positive findings were preserved separately so the report does not equate incompleteness with poor engineering.

Executive Verdict

Nexus contains a substantial and thoughtfully structured actor-system foundation. Its strongest qualities are its package decomposition, immutable behavior model, runtime abstraction, deterministic test runtime, broad observability surface, PSR integration, transactional database appends, and unusually extensive documentation for a pre-1.0 project.

The current product is not production-ready under the claims made in the README and website. This is not a judgment based on missing polish. Several advertised semantics are contradicted by executable code:

- `Effect::none()->thenReply()` silently does not reply.
- Saga callbacks are not replayed after recovery, so the documented crash guarantee is false.
- The documented automatic single-writer protection is disconnected and is not fencing.
- Swoole's "unbounded" mailbox is capped and can report a failed push as accepted.
- Death watch, all-for-one supervision, escalation, exponential backoff, and leaf-first shutdown are incomplete.
- WebSocket upgrades bypass the HTTP authentication and middleware pipeline.
- Class-level HTTP authorization attributes are fail-open when route authorization middleware is omitted.
- Persistence stores default to unrestricted PHP object deserialization.
- TCP cluster delivery and thread-worker delivery can silently drop messages.
- The tagged split-package release model is incompatible with the many retained `dev-main` constraints.
- The committed performance suite is broken, and the audit-time dirty worktree fails Psalm level 1 with 21 errors.

No unconditional Critical-severity remote exploit was established. There are, however, many High-severity correctness and security findings that are release blockers for the workloads Nexus says it supports. Passing unit tests demonstrate a serious engineering baseline, but some tests omit or explicitly encode the broken semantics.

Recommended product position today: experimental actor infrastructure for trusted, non-critical, single-process or carefully constrained single-host use. Do not market it as production-grade, split-brain-safe, durable-saga-capable, lossless, or secure-by-default until the release gates in this report are closed.

Readiness Scorecard

Area	Assessment	Reason
Core architecture	Conditional strength	Good dependency inversion and immutable behavior API, but lifecycle contracts and some package rules are incomplete.
Single-process Fiber/Step use	Usable with constraints	Suitable for learning, tests, prototypes, and non-critical services that tolerate at-most-once delivery and avoid incomplete lifecycle features.
Swoole production runtime	Blocked	Silent mailbox loss, timer cancellation defect, shutdown uncertainty, and unresolved low-traffic worker cycling.
DDD and actor-per-aggregate	Conceptually strong, operationally blocked	Natural single-writer model, but no ownership fencing, command deduplication, durable side-effect model, or event upcasting.
Event sourcing and durable state	Blocked	Effect interpreter defects, misleading recovery semantics, disconnected writer identity, retention mismatch, and synchronous full-history recovery.
HTTP	Conditional	PSR-15 compilation and typed injection are promising; authorization is not secure by default and documented actor shorthand is absent.
WebSockets	Blocked for private/public stateful use	Authentication pipeline bypass and unbounded channel actor churn.
Messenger	Conditional	Explicit registries and type allowlists are good; normal acknowledgement is ack-on-enqueue, not at-least-once processing.
Worker pool	Experimental	Fixed topology, serialized transport, weak overload/liveness model, and silent missing-worker sends.
TCP cluster	Experimental trusted mesh only	Small full mesh, optional security, at-most-once delivery, no Byzantine protection, and no quorum/fencing.
Security posture	Blocked for untrusted or multi-tenant deployment	WebSocket auth bypass, fail-open route authz, native deserialization, DB session reuse, and JWT constraint defects.
Documentation	Broad but unreliable	Excellent coverage and candid thesis material, but multiple flagship examples and guarantees contradict code; snippet verification is not effective CI.
Release engineering	Blocked	No tags, stale release guide, <code>dev-main</code> split dependencies, audit-time Psalm failures, non-blocking coverage/mutation gates, and broken performance suite.

What Nexus Is Good At

- 1. Clear dependency inversion.** Core code depends on small runtime and observability contracts; Fiber, Step, Swoole, worker transport, OpenTelemetry, Doctrine, and persistence adapters are separated.
- 2. Actor-per-aggregate as a DDD fit.** Serial command processing around an identity is a good model for high-contention aggregates when exclusive ownership is actually enforced.
- 3. Immutable behavior model.** `Behavior`, `BehaviorWithState`, immutable messages, and typed `ActorRef<T>` documentation create a coherent programming model.
- 4. Deterministic testing.** `StepRuntime` and virtual time are useful differentiators for state-machine and timer testing.
- 5. HTTP startup compilation.** Reflection and handler resolution are mostly shifted to application compilation rather than repeated per request.
- 6. Persistence adapters.** Database event appends are transactional, and sequence uniqueness catches concurrent sequence races even though it is not writer fencing.
- 7. Bounded TCP ingress controls.** Frame caps, handshake timeouts, link limits, reconnect backoff, nonce replay protection, and tombstones are good defensive ingredients.

8. **Observability breadth.** Metrics, tracing, logging correlation, transport decorators, persistence decorators, and failure isolation show strong operational intent.
9. **Honest conceptual guidance.** The Nexus thesis and actor selection guidance explain where actors are inappropriate better than most framework documentation.
10. **Engineering volume and discipline.** The repository has about 1,372 PHP files, 405 test files, 40 package directories, 265 Markdown/MDX documents across documentation trees, PHPCS, Psalm, Deptrac, PHPUnit, mutation tooling, link checks, and multiple integration environments.

How Nexus Would Really Be Used

1. Local development, teaching, and deterministic tests

This is the strongest current use case. `FiberRuntime` gives a conventional local event loop, and `StepRuntime` gives deterministic stepping and virtual time. Use bounded mailboxes, explicit timeouts, and application-level error reporting. Avoid treating `tell()` as durable delivery.

2. A single-process stateful service

Nexus can coordinate in-memory state or mediate access to a database inside one actor system. This is feasible for internal, non-critical workloads if the service avoids death watch, suspend/resume, advanced supervision, and assumptions about post-commit side effects surviving a crash.

3. An HTTP service with actor-backed dependencies

The viable path is `HttpApp/HttpApplication`, explicit route or global middleware, and `#[FromActor]` injection. The documented `'#actor'` handler shorthand is not implemented. Every protected route must explicitly include authorization middleware today. Do not deploy WebSockets until the separate upgrade path is authenticated and authorized, Origin policy exists, and channel cardinality is bounded.

4. Actor-per-aggregate DDD

The conceptual mapping is strong: one actor serializes commands for one aggregate; pure domain logic decides events; an event handler folds immutable state. The TicTacToe example has the best separation, with pure state and rules outside actor infrastructure.

For real money, inventory, entitlements, or compliance data, applications would still need to build critical infrastructure outside Nexus: durable ownership leases and fencing tokens, expected-version writes, command IDs, deduplication and reply caching, transactional outbox/inbox, idempotent consumers, event type/version registries, upcasters, projection offsets, and recovery budgets.

5. Single-host multi-worker scaling

The worker pool uses consistent hashing and `Swoole\Thread\Queue`, but objects are serialized with `php_serialize`; they are not directly shared without serialization. Topology is static, queue admission and worker health are weak, and a stale worker can become a black hole. Use only for trusted, restart-tolerant, non-durable workloads until transport outcomes, health leases, bounded queues, and recovery ownership are implemented.

6. Broker integration

The Messenger bridge is useful as an adapter between explicit message types and actor targets. The normal consumer acknowledgement occurs after volatile mailbox acceptance, not after handler completion or persistence. Operators must treat it as ack-on-enqueue, add idempotency, and accept a crash-loss window unless Nexus adds process/durable-commit acknowledgement.

7. Multi-machine cluster

The TCP cluster is an experimental, small, trusted full mesh. Delivery is silent at-most-once; security is optional; admitted members have broad control authority; and `minimumMembers` is a downing floor, not quorum. It is not a distributed entity ownership system and does not make persistent aggregates split-brain safe.

Constrained Adopter Journey

The safest current evaluation path is source-based and single-process. It exercises the real runtime and test APIs without relying on unreleased split-package constraints:

```
make build
make up
make install
make test
```

1. Define a narrow protocol as immutable command/reply objects. Add `#[ReplyType(...)]` for Psalm-assisted ask checking, but treat it as static tooling rather than runtime enforcement.
2. Keep decisions and state transitions in ordinary PHP domain objects. Let the actor adapt a command to a domain decision instead of making `ActorContext` part of the aggregate.
3. Build the actor with `Behavior::receive()` or `Behavior::withState()` and `Props::fromBehavior()`. Configure `MailboxConfig::bounded(capacity: ..., strategy: OverflowStrategy::ThrowException)` for every externally fed actor.
4. Spawn through `ActorSystem::create(...)->spawn(...)` and retain the returned typed `ActorRef`; `NexusApp` does not currently return a useful typed root registry.
5. Use `tell()` only where silent fire-and-forget loss is acceptable. The public `ActorRef` contract does not expose the concrete local `offer()` admission result, so application-level admission control must sit before the actor boundary. Every `ask()` needs a finite timeout and an idempotent command.
6. Test state machines with `StepRuntime::step()`, `drain()`, and `advanceTime()`. Test mailbox saturation, timeouts, stop/restart, and duplicate commands explicitly rather than only the happy path.
7. Add observability before load: mailbox depth, accepted/dropped work, handler failures, ask latency/timeouts, persistence latency, and dead-letter counts. Current metric names do not by themselves prove processing or durability.
8. Shut down with an explicit deadline and design resources so parent-first callbacks cannot corrupt child work until leaf-first termination is fixed.

Actor-Per-Aggregate Ownership Boundary

For one process, a stable actor name plus a stable `PersistenceId` can serialize commands for an aggregate. Across workers or hosts, Nexus currently routes to an owner but does not establish durable ownership. The application must own all of the following until the framework supplies them:

Responsibility	Nexus today	Application requirement
Aggregate identity	Actor name and <code>PersistenceId</code>	Stable mapping, validation, and tenant boundary.
Exclusive writer	In-process serialization; sequence conflict detection	Lease/epoch and fencing token enforced by storage.
Retry safety	No command deduplication contract	Command ID, deduplication window, and cached result.
Side effects	Volatile post-commit callback/reply	Transactional outbox/inbox and idempotent consumer.
Event evolution	PHP class name payloads	Stable logical type, schema version, and upcasters.

Responsibility	Nexus today	Application requirement
Projections	No durable projection checkpoint model	Offset store, replay policy, idempotency, and rebuild tooling.
Recovery budget	Synchronous full-history materialization	Snapshot policy, maximum history/latency, timeout, and alerting.

This boundary makes the present product useful as actor infrastructure, but not as a self-contained durable aggregate platform.

DDD Review

Good DDD choices

- Actors naturally provide one-at-a-time command decisions for high-contention aggregate identities.
- Pure event handlers and immutable state values support deterministic replay.
- `PersistenceId` makes aggregate identity explicit.
- Infrastructure adapters are separated into persistence, DBAL, Doctrine, HTTP, Messenger, and cluster packages.
- TicTacToe keeps rules and state transitions independent of the actor runtime.

DDD gaps

- **Identity ownership is assumed, not enforced.** Hashing to a worker is routing, not a durable ownership lease or fencing mechanism.
- **Commands have no framework idempotency model.** Retried financial-style commands can be applied twice after a lost reply.
- **Post-commit effects are not durable.** `thenRun()` and replies happen after commit and are not replayed.
- **Event contracts are PHP class names.** There is no logical event name, schema version, upcaster chain, or unknown-event policy.
- **Projection support is incomplete.** There is no durable offset/checkpoint and replay model for read projections.
- **Saga guidance confuses state recovery with effect recovery.** Reconstructing saga state does not reissue an unjournaled command.
- **Actor plumbing leaks into domain messages.** Many examples put `ActorRef` or reply routing directly in domain protocols. This is acceptable for actor protocols but should not be presented as a pure domain model.
- **Closure signatures are inconsistent.** Stateless, stateful, persistent, durable-state, and Doctrine entity handlers use different argument orders.

DDD conclusion: Nexus can support DDD, but it is not yet a complete DDD platform. Position it as concurrency and application infrastructure. Add ownership, idempotency, effect durability, event evolution, and projection mechanics before claiming durable aggregate or saga guarantees.

DSL and API Review

The core behavior DSL is compact and readable. `Behavior::receive`, `withState`, `setup`, `withTimers`, `withStash`, and `Props` cover useful actor construction patterns. The HTTP DSL has a good compile-time orientation. The main API weakness is semantic overreach: factories and configuration objects expose advanced capabilities that the interpreter does not implement.

Recommended API direction:

1. Make invalid states unrepresentable where possible. If an effect hook is legal, every interpreter path must execute it; otherwise do not expose that chain.
2. Replace raw persistence closures with named handler interfaces or standardized argument order.
3. Replace production `assert()` contract checks with exceptions or type errors.
4. Make `HttpApp::compile()` terminal or cache a compiled instance; reject mutation after compile.
5. Return a `StartedApp` or typed registry from `NexusApp` so callers retain root ports.
6. Describe generics as Psalm-assisted static checking, not PHP runtime type safety.
7. Include the Psalm plugin in the recommended development setup and verify every flagship example with it.

DSL Surface Scorecard

Surface	Ergonomics	Semantic confidence	Current recommendation
Core Behavior	Good: small, immutable, readable	Conditional: lifecycle and unhandled semantics are incomplete	Use for local, bounded, non-critical actors.
Props and mailbox configuration	Good construction API	Conditional: defaults are unbounded and Swoole diverges	Always select a bounded mailbox; load-test the chosen runtime.
StepRuntime	Strong deterministic API	Good for covered single-process state machines	Use as the primary behavior test harness.
NexusApp	Concise startup DSL	Weak composition because spawned refs are discarded	Prefer explicit system composition until a typed started registry exists.
HTTP DSL	Promising compile-time resolution	Conditional: post-compile mutation and authorization gaps	Use explicit middleware and <code>#[FromActor]</code> ; compile once.
Persistence effects	Attractive event-sourcing vocabulary	Blocked: some legal chains are ignored and recovery claims overreach	Do not use for critical durable workflows yet.
Messenger	Clear registry-based bridge	Conditional ack-on-enqueue semantics	Use with application idempotency and an accepted crash-loss window.
Worker pool	Simple hashed routing	Experimental transport and liveness semantics	Keep to trusted, restart-tolerant work.
TCP cluster	Useful low-level primitives	Experimental at-most-once trusted mesh	Do not treat as entity ownership or consensus.
Psalm protocol plugin	Valuable additional assurance	Optional and absent from the meta-package path	Install/configure explicitly and keep analysis blocking.

Consolidated Findings

Severity means:

- **High:** release blocker for a claimed or security-sensitive production use.
- **Medium:** material correctness, operability, or assurance gap with narrower reach or a required deployment condition.
- **Low:** maintainability, clarity, or hardening issue that should be scheduled but does not alone block release.

Finding Index

Family	IDs	High	Medium	Section
DDD	DDD-001 to DDD-005	5	0	Architecture, DDD, Persistence, and DSL
DSL	DSL-001 to DSL-010	3	7	Architecture, DDD, Persistence, and DSL
Architecture	ARCH-001 to ARCH-003	1	2	Architecture, DDD, Persistence, and DSL
Reliability	REL-001 to REL-010	9	1	Runtime, Reliability, Scaling, and Operations
Scaling	SCALE-001 to SCALE-004	3	1	Runtime, Reliability, Scaling, and Operations
Operations	OPS-001 to OPS-005	4	1	Runtime, Reliability, Scaling, and Operations
Security	SEC-001 to SEC-015	5	10	Security
Documentation	DOC-001 to DOC-010	6	4	Documentation and Developer Experience
Quality	QA-001 to QA-006	1	5	Quality and Release Gate Findings
Total	68 findings	37	31	

Architecture, DDD, Persistence, and DSL

DDD-001

HIGH

Interpret every persistence effect hook

FINDING

`Effect::thenReply()` / `thenRun()` store hooks, but `PersistenceEngine::create()` returns unchanged state for `None`; hooks run only through `handlePersist()` (`packages/nexus-persistence/src/EventSourced/Effect.php:127-166`, `packages/nexus-persistence/src/EventSourced/PersistenceEngine.php`). Durable state has the same defect.

IMPACT

Documented read queries time out silently.

REQUIRED ACTION

Centralize effect interpretation and test every base effect/hook combination.

DDD-002

HIGH

Make saga effects crash-recoverable

FINDING

Recovery inside `PersistenceEngine::create()` only folds events; it never reconstructs `thenRun` callbacks (`packages/nexus-persistence/src/EventSourced/PersistenceEngine.php:86-110` , plus post-commit `handlePersist()`), while `website/docs/guides/saga.md:93-97` promises commands are reissued.

IMPACT

A crash after event commit but before side effect permanently stalls a saga or projection.

REQUIRED ACTION

Add a transactional outbox/effect journal or persist pending work explicitly; remove the current guarantee.

DDD-003

HIGH

Fence persistent aggregate writers

FINDING

`ActorSystem::writerId()` is not passed to persistence. Behaviors generate independent ULIDs and replay filtering defaults off (`packages/nexus-core/src/Actor/ActorSystem.php` , `packages/nexus-persistence/src/EventSourced/EventSourcedBehavior.php` , `packages/nexus-persistence/src/EventSourced/PersistenceEngine.php`).

IMPACT

Multiple systems can believe they own one stream. Sequence uniqueness detects some races but does not fence stale writers.

REQUIRED ACTION

Define durable ownership with leases/epochs and fencing tokens plus expected-version writes.

DDD-004

HIGH

Deduplicate wallet commands

FINDING

Wallet commands have no command ID, dedup state, or cached outcome; persistence precedes reply (`examples/nexus-wallet-app/src/Actor/WalletActor.php:80-94`).

IMPACT

A timed-out retry can double-apply a deposit or withdrawal.

REQUIRED ACTION

Add idempotency keys, aggregate deduplication, and reply caching guidance.

DDD-005

HIGH

Version and upcast event schemas

FINDING

PersistenceEngine::handlePersist() sets eventType: \$event::class; EventEnvelope has no schema-version field and no upcaster registry exists (packages/nexus-persistence/src/EventSourced/PersistenceEngine.php:181-185 , packages/nexus-persistence/src/Event/EventEnvelope.php).

IMPACT

Class renames or schema changes can make historical aggregates unrecoverable.

REQUIRED ACTION

Add stable event names, schema versions, ordered upcasters, and compatibility fixtures.

DSL-001

HIGH

Route persistent unhandled commands correctly

FINDING

Persistent Effect::unhandled() maps to unchanged state, despite promising dead-letter routing (packages/nexus-persistence/src/EventSourced/Effect.php , packages/nexus-persistence/src/EventSourced/PersistenceEngine.php; durable-state equivalent).

IMPACT

Protocol drift is silently swallowed.

REQUIRED ACTION

Provide an unhandled transition that reaches actor dead letters or an explicit context API.

DSL-002

MEDIUM

Resolve the HTTP actor-handler contract

FINDING

HTTP docs advertise '#orders', but HandlerResolver supports only closures, Class::method, or invokable classes (packages/nexus-http/src/Handler/HandlerResolver.php:54-70). The implemented actor path is #[FromActor] .

IMPACT

Copied shorthand examples fail with reflection errors.

REQUIRED ACTION

Implement a defined actor HTTP adapter or remove the shorthand and document #[FromActor] .

DSL-003

MEDIUM

Make HTTP compilation terminal and deterministic

FINDING

Repeated HttpApp::compile() rebuilds worker-local actor tables and respawns the same names (packages/nexus-http/src/Dsl/HttpApp.php , packages/nexus-http/src/Actor/ResolvedActorTable.php).

IMPACT

Second compilation throws ActorNameExistsException .

REQUIRED ACTION

Make compilation terminal/idempotent and test every actor mode.

DSL-004**HIGH****Standardize persistence handler signatures****FINDING**

Persistence docs show `(context, command, state)`, but the engine invokes `(state, context, command)` (`packages/nexus-persistence/src/EventSourced/PersistenceEngine.php`; `website/docs/persistence/event-sourcing.md:30-38`).

IMPACT

Flagship examples fail on first command and similar DSLs require different mental models.

REQUIRED ACTION

Standardize signatures and execute documentation examples in CI.

DSL-005**HIGH****Define a real recovery lifecycle****FINDING**

Documentation promises `RecoveryCompleted` and recovery stashing, but no such signal or dispatch exists; recovery is synchronous setup (`website/docs/persistence/event-sourcing.md:82-104`, `packages/nexus-persistence/src/EventSourced/PersistenceEngine.php:86-115`).

IMPACT

Users design lifecycle logic around a nonexistent hook.

REQUIRED ACTION

Document synchronous behavior or implement an asynchronous recovery lifecycle and bounded stash.

DSL-006**MEDIUM****Reject or apply routes added after compile****FINDING**

Routes registered after first compile remain in `pendingBuilders` because promotion occurs only on first compile (`packages/nexus-http/src/Dsl/HttpApp.php`, symbols `compile` and `route promotion`).

IMPACT

Configuration is silently ignored.

REQUIRED ACTION

Reject all mutation after compile or promote on each compile.

DSL-007**MEDIUM****Return typed application root handles****FINDING**

`NexusApp` spawns actors but discards refs; `onStart` gets only `ActorSystem`, with no public named lookup (`packages/nexus-app/src/NexusApp.php:156`).

IMPACT

The app DSL cannot compose typed root ports, so real applications bypass it.

REQUIRED ACTION

Return a typed started-app registry and dependency-aware handles.

DSL-008

MEDIUM

Replace public contract assertions

FINDING

Public factory and reply contracts rely on `assert()` (`packages/nexus-core/src/Actor/Props.php:92,143,163`).

IMPACT

Production with assertions disabled can defer invalid values into obscure failures.

REQUIRED ACTION

Throw explicit contract exceptions; test with `zend.assertions==1`.

DSL-009

MEDIUM

Type-check FromActor parameters at compile time

FINDING

`#[FromActor]` validates name/scope but not that the parameter accepts `ActorRef` (`packages/nexus-http/src/Handler/Resolver/Builtin/FromActorResolver.php`).

IMPACT

Bad handlers compile and fail during invocation.

REQUIRED ACTION

Validate reflected parameter type at compile time.

DSL-010

MEDIUM

Make protocol typing claims reproducible

FINDING

`ActorRef<T>` is a Psalm docblock generic; runtime accepts `object`, the meta-package omits the Psalm plugin, and quick start erases protocols to `object` (`packages/nexus-core/src/Actor/ActorRef.php:41`, `packages/nexus/composer.json:6-12`, `website/docs/getting-started/quick-start.md:70-85`).

IMPACT

"Zero compromises on type safety" is true only with optional tooling and disciplined annotations.

REQUIRED ACTION

Say "Psalm-assisted"; ship plugin config; type-check all examples and `ask/reply` paths.

ARCH-001

HIGH

Make split packages stable-installable

FINDING

Most split manifests retain internal `dev-main`; tag splitting does not rewrite them (`packages/nexus/composer.json:6-12`, `.github/workflows/split.yml:22-88`).

IMPACT

Tagged stable packages remain coupled to moving branch heads and may not resolve for stable consumers.

REQUIRED ACTION

Use `self.version`/release-compatible constraints and install every split package from a clean stable fixture before tagging.

ARCH-002

MEDIUM

Enforce runtime-to-core dependency direction

FINDING

Deptrac permits both Core → Runtime and Runtime → Core while the runtime manifest intentionally has no Core dependency (`deptrac.yaml`).

IMPACT

A future monorepo-only cycle can pass boundary analysis but break split installs.

REQUIRED ACTION

Forbid Runtime → Core and compare imports with Composer dependencies.

ARCH-003

MEDIUM

Decouple Doctrine adapters from HTTP

FINDING

Doctrine DBAL/ORM packages require HTTP packages in their core manifests (`packages/nexus-doctrine-dbal/composer.json` , `packages/nexus-doctrine-orm/composer.json`).

IMPACT

CLI/actor-only consumers receive unnecessary HTTP dependencies and ownership is blurred.

REQUIRED ACTION

Split HTTP middleware/integration into optional packages.

Runtime, Reliability, Scaling, and Operations

REL-001

HIGH

Make Swoole mailbox admission truthful

FINDING

Swoole "unbounded" is a 65,536-slot channel; failed `push()` is ignored and returned as `Accepted` (`packages/nexus-runtime-swoole/src/SwooleMailbox.php:37,74-97`).

IMPACT

Messages and asks can be silently lost under overload.

REQUIRED ACTION

Expose the real capacity, inspect every push result/error, return dropped/backpressured, and fail asks immediately.

REL-002

HIGH

Complete death watch and child cleanup

FINDING

Watchers are recorded but stop never sends `Terminated` ; child maps are never pruned and names cannot be reused (`packages/nexus-core/src/Actor/ActorCell.php:214-249,356-361,681-684`).

IMPACT

Death watch, passivation, child lookup, and respawn contracts are broken.

REQUIRED ACTION

Implement termination propagation, parent deregistration, and nested lifecycle tests.

REL-003**HIGH****Implement advertised supervision strategies****FINDING**

All-for-one never branches on strategy type; `Escalate` becomes stop; backoff parameters are ignored and restart is immediate (`packages/nexus-core/src/Actor/ActorCell.php` , supervision-handling symbols).

IMPACT

Advertised advanced supervision does not behave as configured.

REQUIRED ACTION

Implement parent/sibling propagation and scheduled backoff, or reject unsupported strategies.

REL-004**MEDIUM****Make suspended actors resumable****FINDING**

Suspended actors discard every envelope before system-message dispatch, including `Resume` (`packages/nexus-core/src/Actor/ActorCell.php` ; the gap is encoded by `packages/nexus-core/tests/Unit/Actor/ActorCellAdvancedTest.php`).

IMPACT

Documented resume and priority system messages do not work.

REQUIRED ACTION

Separate system/user queues or process lifecycle messages before the state guard.

REL-005**HIGH****Cancel the active Swoole repeating timer****FINDING**

Repeating Swoole timers return a cancellable for the initial `after` ID, then create a different `tick` ID (`packages/nexus-runtime-swoole/src/SwooleRuntime.php:220-245`).

IMPACT

Cancellation after first fire fails; stopped/restarted actors can retain ticks.

REQUIRED ACTION

Return a handle linked to the real tick and add post-first-fire cancellation tests without runtime shutdown.

REL-006**HIGH****Guarantee leaf-first bounded shutdown****FINDING**

Parent stop enqueues child poison pills, then immediately fires parent `PostStop` ; `ActorSystem` waits only for roots (`packages/nexus-core/src/Actor/ActorCell.php` , `packages/nexus-core/src/Actor/ActorSystem.php`).

IMPACT

Parent resources can close while descendants are still working; shutdown is not leaf-first.

REQUIRED ACTION

Track descendants and await termination inside one shared deadline.

REL-007**HIGH****Define Messenger acknowledgement semantics****FINDING**

Messenger normal consumption acknowledges after mailbox acceptance, before handler execution or persistence (`packages/nexus-messenger/src/Consumer/ReceiverActor.php:358-408`).

IMPACT

Crash after enqueue loses the broker message despite "at-least-once" wording.

REQUIRED ACTION

Rename semantics to ack-on-enqueue or add handler/durable-commit acknowledgement.

REL-008**HIGH****Bound responder-side pending asks****FINDING**

Responder-side pending asks are stored for up to 30 seconds with no bound (`packages/nexus-messenger/src/Consumer/ReceiverActor.php`).

IMPACT

Producers can exhaust consumer memory.

REQUIRED ACTION

Add a configurable cap, load shedding, timeouts, and metrics.

REL-009**HIGH****Expose TCP delivery admission outcomes****FINDING**

TCP disconnected buffers silently drop beyond 100; no route and short writes can disappear while sent metrics increment (`packages/nexus-cluster-tcp/src/PeerConnection.php` , `packages/nexus-cluster-tcp/src/ClusterNode.php` , `packages/nexus-cluster-tcp/src/Swoole/SwoolePeerLink.php`).

IMPACT

Delivery telemetry can claim success for lost messages.

REQUIRED ACTION

Return admitted/buffered/dropped outcomes; document at-most-once; add acknowledgement/retry/dedup if stronger semantics are claimed.

SCALE-001**HIGH****Add worker transport admission and liveness****FINDING**

Thread transport silently ignores missing workers, has no send result/capacity/depth metric, polls with up to 10 ms idle delay, and directory entries do not unregister (`packages/nexus-worker-pool-swoole/src/Transport/ThreadQueueTransport.php:48-118`).

IMPACT

A stale hash-ring worker becomes a black hole; overload is invisible.

REQUIRED ACTION

Add bounded queues, leases/heartbeats, unregister, admission results, and fail-fast pool restart.

SCALE-002**HIGH****Specify and benchmark thread serialization****FINDING**

Thread queue uses `php_serialize` (`packages/nexus-worker-pool-swoole/src/Transport/ThreadQueueTransport.php`) while docs claim direct object passage without serialization.

IMPACT

Resources, closures, PDO, and coroutine primitives cannot cross; throughput depends on payload shape.

REQUIRED ACTION

Validate serializability and benchmark payload sizes; correct documentation.

SCALE-003**HIGH****Stream recovery within explicit budgets****FINDING**

Recovery performs synchronous store I/O and materializes histories; replay filtering buffers all events (`packages/nexus-persistence/src/EventSourced/PersistenceEngine.php` , `packages/nexus-persistence/src/Recovery/ReplayFilter.php`).

IMPACT

Long histories block actor startup and can exhaust memory.

REQUIRED ACTION

Stream/page recovery, add timeouts/budgets/circuit breakers, and require snapshots for large histories.

REL-010**HIGH****Persist post-commit effects****FINDING**

Post-commit replies and commands are volatile and not recovered (`packages/nexus-persistence/src/EventSourced/PersistenceEngine.php` , post-commit effect handling).

IMPACT

Durable state can commit while externally required work is lost.

REQUIRED ACTION

Add transactional outbox/inbox or persisted pending effects.

OPS-001**HIGH****Bound and unify dead-letter handling****FINDING**

`DeadLetterRef` retains every object forever, while some closed-mailbox paths bypass dead letters (`packages/nexus-core/src/Actor/DeadLetterRef.php` , `packages/nexus-core/src/Actor/LocalActorRef.php`).

IMPACT

Memory grows and delivery failures are inconsistently observable.

REQUIRED ACTION

Bound retained samples, keep counters, emit events, and route all failure paths consistently.

OPS-002

HIGH

Implement snapshot retention semantics

FINDING

`keepSnapshots` is not applied; snapshots are not pruned and events are deleted through the newest snapshot (`packages/nexus-persistence/src/EventSourced/RetentionPolicy.php`, `packages/nexus-persistence/src/EventSourced/PersistenceEngine.php`).

IMPACT

Snapshot storage grows and event history can be deleted more aggressively than documented.

REQUIRED ACTION

Implement oldest-retained-snapshot semantics and adapter integration tests.

OPS-003

MEDIUM

Reproduce and control Swoole worker cycling

FINDING

Operations docs record unexplained Swoole worker cycling and cases where `WorkerStop` does not fire; shutdown timeout is not mapped to Swoole `max_wait_time` (`website/docs/operations/swoole-deadlock-detector.md:40-74`). This is an unverified operational risk rather than a reproduced framework failure.

IMPACT

If reproduced in deployment, in-memory state/WebSocket sessions can be lost during restarts.

REQUIRED ACTION

Root-cause the cycling, configure native shutdown, and run restart/soak tests before production claims.

OPS-004

HIGH

Separate membership floors from write safety

FINDING

`minimumMembers` suppresses a downing transition; it is not quorum or write fencing (`packages/nexus-cluster-tcp/src/Membership/MembershipService.php:343-405`).

IMPACT

Both partition sides can keep serving and writing.

REQUIRED ACTION

Rename the setting and require external consensus/lease fencing for stateful ownership.

SCALE-004

MEDIUM

Publish multi-host cluster limits

FINDING

TCP cluster is $O(N^2)$; only a 16-node result is cited, with around 50 described as comfortable and redesign beyond 100 (`packages/nexus-cluster-tcp/README.md:52-61`).

IMPACT

The cited topology does not establish multi-host latency, churn, TLS, packet-loss, or partition behavior.

REQUIRED ACTION

Treat 16 as demonstrated only under its published setup, 50 as hypothesis; add multi-host chaos/soak tests.

OPS-005**HIGH****Restore reproducible performance harnesses****FINDING**

Published cluster benchmark docs reference absent harnesses; the committed performance suite fatally references removed

`ConsistentHashRing`
(`tests/Performance/ClusterPerformanceTest.php:64`).

IMPACT

Throughput claims are not independently reproducible.

REQUIRED ACTION

Repair harnesses, publish environment/raw data, and make performance regression jobs executable.

Security**Attack surface**

- Internet → Swoole HTTP translation → PSR-15 router and handlers.
- Internet → Swoole WebSocket `open` → separate WebSocket dispatcher.
- Broker producer → Messenger decoder/type registry → receiver actor → configured target actor.
- Cluster socket → frame codec → optional TLS/HMAC → membership/control plane and exposed actors.
- Persistence database → serializer → actor recovery.
- Pooled DB connection → later request, actor, or tenant.
- Repository contributor/dependency registries/GitHub Actions → build artifacts, split repositories, and documentation deployment.
- Health, logs, metrics, and traces → operators and potentially public callers.

SEC-001**HIGH****Authorize WebSockets before upgrade****FINDING**

Swoole WebSocket `open` translates and dispatches directly, bypassing HTTP/auth middleware (`packages/nexus-http-server-swoole/src/Bridge/SwooleServerEventBinder.php:77-95`). WS compilation constructs its own handler registry without shared principal resolution (`packages/nexus-http-ws/src/WebSocket/HandlerInstantiator.php:123-140` , `packages/nexus-http-ws/src/WsApplication.php`).

IMPACT

WebSocket routes documented with auth can be reached without that pipeline; principal injection is not wired.

REQUIRED ACTION

Authenticate/authorize in pre-upgrade handshake, add WS route middleware and resolver sharing, reject before 101, and add real Swoole token tests.

SEC-002**HIGH****Bound and passivate WebSocket channel actors****FINDING**

Arbitrary channel keys create actors; registry entries are only pruned on future lookup of the same key, `remove()` is unused, base actors stay alive on close, and mailboxes default unbounded

(`packages/nexus-http-ws/src/WebSocket/WebSocketDispatcher.php` , `packages/nexus-http-ws/src/WebSocket/ChannelActorRegistry.php` , `packages/nexus-http-ws/src/WebSocket/WebSocketChannelActor.php`).

The High rating applies to public non-passivating routes; strict authentication, cardinality limits, and correct application stop behavior reduce exposure.

IMPACT

Repeated unique paths exhaust actors, refs, mailboxes, and memory; SEC-001 removes the expected auth barrier.

REQUIRED ACTION

Stop on last close by default, evict on lifecycle signal, cap/TTL/LRU channel cardinality, bound mailboxes, validate keys, and rate-limit.

SEC-003**HIGH****Make annotated HTTP authorization fail closed****FINDING**

`#[RequiresAuth]` / `#[RequiresScope]` are inspected only by `AuthorizationMiddleware`; compile does not enforce it and authentication allows anonymous (`packages/nexus-http-auth/src/Middleware/AuthorizationMiddleware.php` , `packages/nexus-http-auth/src/Middleware/AuthenticationMiddleware.php` , `packages/nexus-http/src/Dsl/HttpApp.php`).

IMPACT

A protected class serves HTTP 200 if route authz middleware is omitted.

REQUIRED ACTION

Auto-install authorization after routing or fail application compilation for unprotected annotated routes.

SEC-004**MEDIUM****Replace default native object deserialization****FINDING**

`PhpNativeSerializer` defaults to allow-all and invokes `unserialize()` before top-level checks; DB persistence adapters instantiate it by default (`packages/nexus-serialization/src/PhpNativeSerializer.php:20-45,68-100` , `packages/nexus-persistence-dbal/src/DbalEventStore.php:20-25`). Safer serializers and allowlists exist but are not the default. If an attacker or operator can influence persisted rows, gadget construction can become High-impact RCE.

IMPACT

Recovery crosses an unsafe native object-deserialization boundary.

REQUIRED ACTION

Default to schema codecs and explicit type registries; make native deserialization an explicit trusted-data opt-in with nested allowlists.

SEC-005**MEDIUM****Sanitize pooled database connections****FINDING**

DB connection release requeues without rollback or session reset (`packages/nexus-doctrine-dbal/src/Pool/ConnectionPool.php:91-131`); ORM reuses the underlying connection. Multi-tenant or session-role deployments make the impact High.

IMPACT

A later request can inherit an active transaction, role, search path, locks, or tenant state.

REQUIRED ACTION

Roll back while active, reset session state or evict, poison on cleanup failure, and partition tenant pools.

SEC-006**HIGH****Enforce JWT issuer and audience constraints****FINDING**

JWT validation asserts signature and time but ignores configured issuer/audience constraints (`packages/nexus-http-auth/src/Authenticator/JwtAuthenticator.php:61-78,103-108`). Shared signing keys make cross-issuer/service replay directly exploitable.

IMPACT

Correctly signed tokens for another issuer/service/tenant can be replayed.

REQUIRED ACTION

Merge configured constraints and require/test issuer, audience, subject, and clock skew.

SEC-007**HIGH****Secure cluster production defaults****FINDING**

Cluster topology defaults TLS and authentication to null (`packages/nexus-cluster-tcp/src/ClusterTopology.php:76-158`). The exposure requires a reachable insecure production deployment, but that is an allowed default configuration.

IMPACT

Any reachable peer can join and send control/actor traffic when production config omits security.

REQUIRED ACTION

Add secure production factory; reject non-loopback insecure bind unless explicit development override.

SEC-008**MEDIUM****Authorize cluster node identities and control****FINDING**

An admitted cluster member can steer endpoints/control events and address exposed actors; shared HMAC proves group secret possession, not node identity (`packages/nexus-cluster-tcp/src/ClusterNode.php`).

IMPACT

A malicious member has broad data/control authority, including endpoint steering and leave messages.

REQUIRED ACTION

Bind per-node certificates to node identity, allowlist endpoints, authorize control events, and segment trust domains.

SEC-009

MEDIUM

Enforce request limits before allocation

FINDING

Swoole materializes `rawContent()` before PSR body middleware; body limit skips unknown lengths and GET bodies (`packages/nexus-http-server-swoole/src/Bridge/SwooleRequestTranslator.php:31-56`, `packages/nexus-http-toolkit/src/Middleware/BodySizeLimitMiddleware.php:61-100`).

IMPACT

Request-memory exhaustion occurs before framework limit enforcement.

REQUIRED ACTION

Configure native Swoole size/header/time limits, count streams, check all methods, and add edge concurrency/rate limits.

SEC-010

MEDIUM

Prevent cross-site WebSocket and cookie attacks

FINDING

The WebSocket open/dispatch path does not validate Origin, and `CookieTokenExtractor` only reads a cookie value (`packages/nexus-http-server-swoole/src/Bridge/SwooleServerEventBinder.php`, `packages/nexus-http-ws/src/WebSocket/WebSocketDispatcher.php`, `packages/nexus-http-auth/src/Extractor/CookieTokenExtractor.php`). Exploitation requires browser-carried credentials such as cookie bearer tokens.

IMPACT

Cross-site WebSocket hijacking and HTTP CSRF are possible with cookie bearer tokens.

REQUIRED ACTION

Exact Origin allowlist, SameSite cookies, CSRF token, and rejection tests.

SEC-011

MEDIUM

Separate public liveness from private readiness

FINDING

Health handler returns check details plus exception class/message; docs show a public `/health` route (`packages/nexus-http-toolkit/src/Health/HealthCheckHandler.php:49-59`).

IMPACT

Internal topology, component, or error details can be disclosed.

REQUIRED ACTION

Separate opaque liveness from authenticated/internal readiness and redact log/health errors.

SEC-012

MEDIUM

Authorize Messenger producer-to-target routes

FINDING

Messenger uses strong type/target allowlists, but producer-controlled stamps select configured actor targets and provenance metadata without per-message origin authorization (`packages/nexus-messenger/src/Router/StampMessageRouter.php:28-38`).

IMPACT

A producer with publish rights can invoke any registered target and consume capacity.

REQUIRED ACTION

Enforce broker ACLs and message-to-target authorization; authenticate envelopes across mutually untrusted producers.

SEC-013

MEDIUM

Harden wallet example administration

FINDING

Wallet demo exposes `/admin/wallets` without auth and publishes demo DB/token defaults (`examples/nexus-wallet-app/src/Http/WalletRoutes.php:84-91` , `examples/nexus-wallet-app/README.md:97-100` , `examples/nexus-wallet-app/compose.yaml:17-40,74-83`).

IMPACT

Copied deployment leaks every owner/balance and exposes demo infrastructure.

REQUIRED ACTION

Add admin role/authz, loopback-bind or unpublish DB, and fail production mode on demo defaults.

SEC-014

MEDIUM

Harden release supply-chain inputs

FINDING

PHP lockfile is ignored; CI runs no dependency audits; split workflow downloads an unchecked binary and executes it with a split PAT; actions/forks use mutable tags/branches (`.gitignore:3` , `.github/workflows/split.yaml:67-88`).

IMPACT

Non-reproducible dependencies and compromised build inputs can affect all split repos/releases.

REQUIRED ACTION

Commit/review lock policy, add Composer/npm/OSV audits, pin SHAs, verify checksums, narrow permissions/PAT, and publish SBOM/provenance.

SEC-015

MEDIUM

Run security packages in primary CI

FINDING

Root `unit` suite omits HTTP auth/toolkit directories (`phpunit.xml:9-39`); CI invokes only named root suites.

IMPACT

Security packages can regress while primary CI stays green.

REQUIRED ACTION

Include both suites and add explicit negative security regression tests.

Security strengths

- Production HTTP error mode is opaque by default.
- JWT signature and strict time validation are present; the gap is issuer/audience configuration.
- Cluster frames are length-bounded before body buffering.
- HMAC uses SHA-256, fresh random nonces, a replay cache, and `hash_equals`.
- Cluster actor exposure, Messenger target maps, and Messenger type registries are explicit allowlists.
- Database queries reviewed were parameterized.
- `PhpNativeSerializer` can operate in allowlist mode even though unsafe allow-all is the default.
- WebSocket compression is disabled by default.
- `SECURITY.md` provides private reporting, acknowledgement expectations, and a disclosure policy.
- The current Composer dependency snapshot had no known advisories at audit time.

Documentation and Developer Experience

Strengths

- The site covers concepts, runtimes, persistence, HTTP, WebSockets, Doctrine, Messenger, operations, observability, scaling, ADRs, reference, tutorials, and contribution workflows.
- The thesis and "when to use actors" guidance are useful and appropriately skeptical.
- ADRs document major decisions and alternatives.
- Docusaurus lint and production build pass.

Findings

DOC-001**HIGH**

Make the wallet guide executable and safe

FINDING

Wallet README describes removed `RequestActor` files, sends `amount` while DTO requires `amountCents`, and defaults each worker to a separate `InMemoryEventStore` (`examples/nexus-wallet-app/README.md:40-80,203-235`, `examples/nexus-wallet-app/src/Boot/WalletApp.php:129-134`, `examples/nexus-wallet-app/src/Http/Request/AmountRequest.php:8-16`).

IMPACT

Adopters can copy a financially themed example that uses stale files, invalid payloads, and unsafe per-worker state.

REQUIRED ACTION

Make it single-worker or shared persistent/affinity-safe; fix payloads and file map; label financial limitations.

DOC-002

HIGH

Align event-sourcing documentation with code

FINDING

Event sourcing docs show the wrong handler order, nonexistent `RecoveryCompleted`, working `none()->thenReply`, side effects after `none`, and automatic system writer ID (`website/docs/persistence/event-sourcing.md:30-63,82-157`).

IMPACT

The flagship persistence guide teaches APIs and guarantees that fail or do not exist.

REQUIRED ACTION

Correct docs only after the underlying contracts are fixed or explicitly document current semantics.

DOC-003

HIGH

Replace the saga guide with an executable model

FINDING

Saga guide is skipped from verification and contains wrong handler order, uncaptured refs in a static closure, and false replay/reissue guarantee (`website/docs/guides/saga.md:16-97`).

IMPACT

The current guide can neither execute as written nor deliver its promised crash recovery.

REQUIRED ACTION

Replace with an executable outbox/pending-intent example.

DOC-004

MEDIUM

Describe replay filtering without false repair claims

FINDING

`RepairByDiscardOld` docs say events are permanently dropped; implementation only filters the current in-memory replay list (`website/docs/persistence/single-writer.md:64-75`, `packages/nexus-persistence/src/Recovery/ReplayFilter.php`).

IMPACT

Operators may believe corrupt or conflicting stored events were permanently repaired when only one replay was filtered.

REQUIRED ACTION

Describe exact behavior and remove "repair" language until storage mutation is explicit.

DOC-005

MEDIUM

Publish one generated Swoole requirement

FINDING

Swoole requirements disagree: installation says 5.0+, the root README says 6.0+, and runtime/server manifests require 6.2.1+ (`website/docs/getting-started/installation.md:18`, `README.md:107,155`, `packages/nexus-runtime-swoole/composer.json`, `packages/nexus-http-server-swoole/composer.json`).

IMPACT

Conflicting version requirements cause failed installs and make the supported platform impossible to determine.

REQUIRED ACTION

Generate requirements from package constraints and test install matrices.

DOC-006

HIGH

Fix documentation snippet bootstrapping

FINDING

Documentation verifier prepends `<?php` to every snippet even when the snippet already contains `<?php` (`bin/verify-doc-snippets:113-121`). Of 580 PHP fences, 83 contain open tags and 59 are full-verification fences; the exact concatenation fails lint with an unexpected `<`.

IMPACT

A broken verifier gives false confidence and rejects valid tagged examples before analysis.

REQUIRED ACTION

Strip an existing open tag or do not prepend one; add verifier self-tests and a fast batch mode.

DOC-007

HIGH

Make snippet verification a practical CI gate

FINDING

`docs-verify` is not called by any GitHub workflow. It would run Psalm with `--no-cache` separately for about 480 snippets, making it impractical as a gate (`bin/verify-doc-snippets:132-151` ; `.github/workflows/pages-docs.yml:31-33`).

IMPACT

Hundreds of uncached analyzer processes keep documentation correctness outside required CI.

REQUIRED ACTION

Batch generated snippets into one analysis project, cache Psalm, and make verification required.

DOC-008

MEDIUM

Document every split package locally

FINDING

Only 23 of 40 package directories have package-local READMEs; 17 important packages rely solely on central docs (BASE tree count reproduced below).

IMPACT

Seventeen split packages lack the local information consumers expect on package registries.

REQUIRED ACTION

Generate or maintain minimal install/usage/compatibility READMEs for every split package.

DOC-009

HIGH

Generate release documentation from manifests

FINDING

Release guide says 14 packages and `^1.0` internal constraints, while split matrix contains 37 entries and manifests mostly use `dev-main` (`website/docs/contributing/release-process.md:11,22-55`).

IMPACT

Maintainers following the stale guide can publish an incomplete or unresolvable release.

REQUIRED ACTION

Rewrite release documentation from executable manifest/matrix checks.

DOC-010

MEDIUM

Define a genuinely comprehensive test command

FINDING

Root `make test` is described as all tests but excludes Swoole, real cluster, Doctrine Swoole, HTTP Swoole, and performance suites (Makefile:25-66, website/docs/getting-started/installation.md:88-96).

IMPACT

Contributors can receive a green "all tests" result while major runtimes and integration suites never ran.

REQUIRED ACTION

Rename it or create a truly comprehensive aggregate target.

README count reproduction:

```
git ls-tree -d --name-only 8970faa2:packages | wc -l
git ls-tree -r --name-only 8970faa2:packages \
  | awk -F/ 'NF == 2 && $2 == "README.md" { count++ } END { print count }'
# 40 package directories; 23 package-local README.md files
```

Quality and Release Gate Findings

QA-001

MEDIUM

Make risk-based coverage thresholds blocking

FINDING

Coverage is collected, but the coverage guard is `continue-on-error: true` with a TODO for central actor internals (.github/workflows/ci.yml:128-144).

IMPACT

Coverage regressions in the highest-risk runtime paths do not currently block merges.

REQUIRED ACTION

Define risk-based thresholds and make the guard blocking after closing the stated core gaps.

QA-002

MEDIUM

Restore blocking mutation assurance

FINDING

Mutation testing is also `continue-on-error: true` (.github/workflows/ci.yml:209-240).

IMPACT

Tests can exercise lines without detecting behavior regressions, while the mutation job is allowed to fail.

REQUIRED ACTION

Restore compatible mutation tooling and make an agreed mutation score blocking for core/persistence/security packages.

QA-003

HIGH

Make performance smoke tests executable

FINDING

Current performance tests are neither a reliable benchmark gate nor executable as committed;
`tests/Performance/ClusterPerformanceTest.php:64` references removed `ConsistentHashRing`.

IMPACT

The committed suite cannot detect performance regressions or substantiate published capacity claims.

REQUIRED ACTION

Separate correctness performance assertions from benchmark jobs and require a smoke run on every relevant change.

QA-004

MEDIUM

Include auth and toolkit suites in required CI

FINDING

Primary CI omits HTTP auth/toolkit test directories even though those packages protect external input (`phpunit.xml:9-39`, `.github/workflows/ci.yml`).

IMPACT

External-input security code can regress while the primary named suites remain green.

REQUIRED ACTION

Add the directories to named suites and run negative security cases against real adapters.

QA-005

MEDIUM

Restore Psalm level 1 on an immutable commit

FINDING

The audit-time dirty worktree fails Psalm with 21 errors while README/product claims emphasize Psalm level 1. This result must not be attributed to clean `origin/main` because concurrent uncommitted work appeared during the audit.

IMPACT

The audit-time tree cannot substantiate its level-1 type-safety claim while static analysis reports errors.

REQUIRED ACTION

Finish the suppression-removal work, run Psalm on an immutable commit, and keep it blocking in CI.

QA-006

MEDIUM

Cover first-party symbols in architecture checks

FINDING

Deptrac reports 0 violations but 1,023 uncovered tokens, and its rules allow at least one package-direction mismatch (`deptrac.yaml`; command and output summarized in [Verification Record](#)).

IMPACT

A zero-violation report can conceal more than one thousand unclassified symbols and package-direction mistakes.

REQUIRED ACTION

Classify uncovered project symbols, fail on uncovered first-party namespaces, and align rules with Composer manifests.

Verification Record

Commands were run against the DIRTY working tree in the repository's existing Docker environment. The pre-existing untracked `.codegraph/` and `.whet/` directories were not intentionally modified. Results below are verification evidence, not claims about BASE.

Verification	Result
<code>make test</code>	Passed: 1,741 tests, 5,323 assertions, 12 notices, 8 skipped, 3m42s. Recurring OpenTelemetry fiber-context warnings were emitted.
Independent core/unit run	Passed: 1,585 tests, 4,713 assertions, 12 notices, 8 skipped.
Swoole unit	Passed: 86 tests.
Persistence unit	Passed: 143 tests.
Messenger integration	Passed: 24 tests.
Cluster loopback integration	Passed: 36 tests.
Worker-pool Swoole integration	Passed: 2 tests.
Persistence integration	Passed: 11 tests.
Omitted auth/toolkit directories	Passed when run manually: 77 tests, 150 assertions. They do not cover the security findings above.
<code>make psalm</code>	Failed on the audit-time dirty worktree: 21 errors at Psalm level 1, including generic return types, unused code, untyped actor refs, observability props typing, and skeleton analysis. This does not establish the status of clean origin/main.
<code>make phpcs</code>	Passed: 930 files.
Deptrac	Passed with 0 violations and 0 skipped violations; report included 1,023 uncovered tokens.
<code>composer validate --strict</code>	Passed.
<code>composer audit --locked</code>	Passed at audit time: no known advisories in the local PHP dependency snapshot. Root lockfile is not committed.
Website ESLint	Passed.
Website Docusaurus build	Passed.
Website dependency audit	Reported 1 high and 24 moderate affected entries in build tooling; landing audit was clean.
Documentation snippet verifier	Full run not treated as valid evidence: verifier is not in CI, has duplicate-open-tag failures, and starts hundreds of uncached Psalm processes. Exact bootstrap reproduction failed as predicted.
Performance suite	Failed fatally: <code>tests/Performance/ClusterPerformanceTest.php:64</code> references removed <code>Monadial\Nexus\Cluster\ConsistentHashRing</code> .

Assurance gaps not covered by passing tests

- Packet loss, partitions, asymmetric links, slow peers, and reconnect ordering.
- Disk full, corrupt event, database outage/latency, and snapshot/event race recovery.
- Broker outage, redelivery, restart between enqueue and execution, and duplicate commands.
- Swoole restart storms, long low-traffic soak, and nested actor shutdown ordering.
- Mailbox saturation at 65,536, queue admission, and memory pressure.
- Multi-host/TLS cluster load and chaos.
- WebSocket handshake auth, Origin validation, unique-key churn, and actor eviction.
- Missing route authorization as a compile-time failure.
- DB transaction/role/search-path cleanup across sequential tenants.
- PHP native deserialization from tampered persisted rows.
- Stable split-package installation from Packagist-like repositories.

Marketing and Claim Corrections

Current claim	Evidence-based wording now
"Production-grade actor system"	"Pre-1.0 actor-system toolkit under active development."
"Zero compromises on type safety"	"Psalm-assisted protocol checking when the optional plugin and concrete annotations are enabled."
"Supervision that actually works"	"Basic one-for-one directives and retry windows are implemented; all-for-one, escalation, backoff, and failure signals are incomplete."
"Two runtimes, one API"	Core actor code is mostly portable, but mailbox capacity, cancellation, shutdown, and observability semantics currently differ.
"No application-level serializer required"	No user-supplied serializer is required for thread queues, but Swoole uses PHP serialization internally.
"260K msgs/sec"	Label as a specific local benchmark with hardware, payload, topology, raw harness, and reproducible command; current performance suite is broken.
"Single-writer guarantee"	"Application must ensure one owner; current sequence/version conflicts detect some races but Nexus does not lease or fence writers."
"Saga survives crashes and reissues work"	"Saga state replays; post-commit commands require an application outbox/pending-intent mechanism and idempotent delivery."
"At-least-once Messenger"	"Normal consumption acknowledges after mailbox enqueue; a crash before processing can lose work."
"Unbounded mailbox"	"Large fixed-capacity mailbox in Swoole" until genuine overflow behavior is defined.
"Authenticated WebSockets use HTTP middleware"	Remove until authentication/authorization occurs before the upgrade.

Prioritized Remediation Roadmap

Release containment (parallel, not remediation)

Run these containment actions immediately and in parallel with implementation work. They reduce adopter exposure but do not close technical findings. A technical finding closes only when implementation, tests, and documentation land together, or when the affected feature is deliberately removed; documentation-only wording changes are not remediation.

1. Remove or qualify production-grade, durable saga, automatic single-writer, lossless unbounded mailbox, advanced supervision, WebSocket auth, and at-least-once processing claims.
2. Put explicit experimental warnings on worker-pool, cluster, persistence, wallet, WebSocket, and benchmark pages.
3. Repair the release guide and state that no stable release should be cut from current manifests.

Phase 1: Close direct correctness and security blockers

1. Fix effect interpretation for `None`, `Unhandled`, hooks, reply, stop, and durable state with a complete matrix test.
2. Authenticate and authorize WebSockets before upgrade; add Origin policy and route middleware.
3. Make protected HTTP annotations fail compilation without authorization.
4. Fix Swoole mailbox admission, ask failure, and transport outcome propagation.
5. Fix repeating timer cancellation.
6. Reset/rollback pooled DB connections on every return.
7. Enforce JWT issuer/audience constraints.
8. Replace default native persistence deserialization with a registry/schema codec.

Phase 2: Make lifecycle and delivery contracts real

1. Implement death watch, `Terminated`, `ChildFailed`, parent deregistration, and name reuse.
2. Implement or remove all-for-one, escalation, exponential backoff, suspend/resume, and leaf-first shutdown.
3. Bound dead letters, WebSocket channel registries, responder asks, exposed actors, and all external-input mailboxes.
4. Define one admission/result vocabulary across local, worker, Messenger, and cluster delivery.
5. Make metrics count accepted, buffered, dropped, executed, persisted, and acknowledged events separately.

Phase 3: Build durable DDD infrastructure

1. Add durable ownership lease/epoch and fencing tokens with expected-version writes.
2. Add command IDs, deduplication windows, idempotency keys, and cached outcomes.
3. Add transactional outbox/inbox or a persisted effect journal.
4. Add event logical names, schema versions, upcasters, unknown-event policy, and compatibility fixtures.
5. Add projection offsets, replay, idempotent handlers, and operational rebuild tooling.
6. Stream/page recovery with budgets, timeouts, snapshots, and latency metrics.
7. Correct snapshot/event retention semantics.

Phase 4: Prove scaling and operations

1. Add bounded worker queues, liveness leases, unregister, static-topology failure policy, and resharding documentation.
2. Publish payload-size and serializer benchmarks, not only empty/small-message throughput.

3. Add multi-host cluster tests with TLS, packet loss, asymmetric partitions, churn, slow peers, and reconnects.
4. Separate cluster membership from entity ownership; use external consensus/leases for stateful activation.
5. Root-cause Swoole cycling and prove graceful restart under long-lived actors and WebSockets.
6. Repair the performance suite and publish reproducible raw results.

Phase 5: Establish a releasable supply chain

1. Make Psalm level 1, coverage thresholds, and mutation thresholds blocking.
2. Add auth/toolkit/security regression suites to required CI.
3. Repair and require documentation snippet verification.
4. Validate every split package and meta-package in isolated stable Composer fixtures.
5. Replace `dev-main` with release-compatible internal constraints.
6. Pin third-party actions/tools, verify splitsh artifacts, narrow tokens, and publish SBOM/provenance.
7. Define version support, upgrade/BC policy, release candidate soak, and rollback procedure.

Release Gates

Do not call Nexus production-ready until all of the following are true:

- Every High finding in every family (DDD, DSL, architecture, reliability, scaling, operations, security, documentation, and quality) is closed, the corresponding feature or claim is removed, or a named accountable owner signs a time-bounded risk acceptance with deployment controls. General acceptance of "experimental" status is not sufficient.
- Actor lifecycle/supervision documentation is covered by cross-runtime contract tests.
- WebSocket and HTTP negative authorization tests pass against real Swoole.
- Persistent aggregates have documented fencing, idempotency, outbox, event evolution, and recovery limits.
- Delivery semantics and failure outcomes are explicit for local, thread, broker, and TCP paths.
- The full test/static-analysis/style/dependency/doc/performance pipeline is green with blocking gates.
- Tagged split packages install independently from stable-only clean projects.
- At least one multi-host soak/chaos campaign and one long-running Swoole restart campaign have published results.

Final Feasibility Assessment

The project is technically feasible. PHP 8.5, Fibers, Swoole coroutines/threads, Psalm plugins, PSR interfaces, Doctrine adapters, and explicit serialization can support a useful actor framework. The repository demonstrates enough implementation depth to justify continuing.

The largest demonstrated feasibility risk is semantic scope. Nexus is simultaneously attempting actor lifecycle, two local runtimes, thread scaling, TCP clustering, event sourcing, durable state, HTTP, WebSockets, auth, Messenger, Doctrine pooling, observability, a Psalm type system, package splitting, and production operations. Several features have complete-looking APIs and documentation but partial interpreters. Independent throughput, latency, memory, and operating-cost limits remain unquantified because the committed performance suite is broken.

The pragmatic path is to narrow the first stable release:

1. One complete local runtime contract, with Fiber and Step as the release core.
2. Correct bounded delivery, lifecycle, basic supervision, and deterministic testing.
3. HTTP without WebSockets until authentication, authorization, Origin validation, and channel-cardinality controls close SEC-001, SEC-002, and SEC-010.
4. Persistence only after fencing, idempotency, durable effects, and event evolution exist.
5. Swoole, worker pool, and TCP cluster graduated separately behind explicit experimental maturity labels and evidence gates.

With that reduction and the roadmap above, Nexus can become a credible PHP actor toolkit. Without it, the breadth of the public promise will continue to outrun the semantics that production users depend on.

Evidence Appendix

Baseline and Working-Tree Fingerprint

At 2026-07-16T12:37:28+02:00, HEAD was 8970faa20ae0. The concurrently changing tree had 201 porcelain status entries. Its status-list SHA-256 was ce54be1d8b343adde72e9cf5d9125f8d9cd4f1d0d2f65d46ad6f04ab7180ae98; its tracked binary-diff SHA-256 was 4f33be3dbe532613ca6b5e67da713cb9af5003e2e42a5ba468cd2e14a5bd5b0f. These fingerprints include unrelated user work and the audit document; they identify the observation point rather than a releasable tree.

Reproduction Commands

```
# Immutable finding evidence
git show 8970faa2:<repository-relative-path> | nl -ba
git diff 8970faa2 -- <repository-relative-path>

# DIRTY verification gates
make test
make psalm
make phpcs
docker compose exec -T php composer validate --strict
docker compose exec -T php composer audit --locked --no-interaction
docker compose exec -T php vendor/bin/deptrac analyse --no-progress
npm --prefix website run lint
npm --prefix website run build

# Working-tree observation fingerprint
git status --porcelain=v1 | shasum -a 256
git diff --binary | shasum -a 256
```

Specialized PHPUnit suites were also run directly inside the existing PHP/Swoole containers, as recorded in the Verification Record. Security reviewers used focused, ad hoc runtime probes for missing route authorization and JWT constraint behavior; those probes were not committed. The cited source paths independently expose the same control flow, but future remediation should add permanent negative regression tests.

Audit Limitations

- This was a source-assisted product/security review, not a formal certification, legal compliance assessment, or destructive penetration test.
- Deployment-specific controls such as network policies, broker ACLs, database roles, IdP key reuse, GitHub organization controls, and production Swoole configuration were unavailable.
- Absolute throughput conclusions are limited because the committed performance suite is broken and no independent multi-host load environment was provisioned.
- Dependency advisory results are time-sensitive and reflect the snapshots queried on the audit date.
- Semantic findings refer to BASE commit 8970faa2; verification results refer to the timestamped DIRTY worktree. Line numbers may move, so reproduce by path and symbol as described in Evidence Provenance.